

# Manejo e Implementación de Archivos



Semana #6

Guatemala, 12 de agosto de 2026

Ing. Luis Aguilar



# Objetivos

---

Al finalizar la clase el estudiante será capaz de:

- Comprender la organización y acceso de archivos
- Identificar tipos de organización: pila, secuencial, indizado
- Analizar métodos de acceso: secuencial, directo, búsqueda
- Aplicar conceptos en casos empresariales



# ORGANIZACIÓN DE ARCHIVOS

## ACCESO SECUENCIAL



El acceso secuencial consiste en leer los registros **uno tras otro**, en el orden en que están almacenados, desde el inicio hasta el final.



# 1

## EJEMPLO REAL

### ACCESO SECUENCIAL EN UN ARCHIVO DE TEXTO



Archivo de texto (CSV)

Ejemplo: alumnos.txt

```
alumnos.txt
1 Ana Pérez,18,Sistemas
2 Carlos López,19,Industrial
3 Luis Martínez,20,Mecatrónica
4 María Gómez,21,Sistemas
5 Pedro Soto,22,Electrónica
6 Julia Ramos,23,Industrial
...
```

#### ¿CÓMO FUNCIONA?

El sistema lee los registros uno por uno, desde el inicio del archivo hasta encontrar el que cumple la condición o llegar al final.



Buscamos: "María Gómez"

El sistema lee: 1 → 2 → 3 → 4 ✓ (encontrado en el 4° registro)



#### IDEA CLAVE

El acceso secuencial lee los registros uno por uno, en el orden en que están almacenados, desde el inicio hasta encontrar el buscado o llegar al final.





## 2

# PSEUDOCÓDIGO

## ACCESO SECUENCIAL EN UN ARCHIVO DE TEXTO

Objetivo: Buscar un registro por nombre en un archivo de texto leyendo secuencialmente.

### PSEUDOCÓDIGO

```
1 Inicio
2   Abrir archivo para lectura como archivo
3   Escribir "Ingrese el nombre a buscar: "
4   Leer nombreBuscado
5   encontrado ← FALSO
6   numeroRegistro ← 0
7   Mientras NO EOF(archivo) Y NO encontrado Hacer
8     Leer línea desde archivo
9     numeroRegistro ← numeroRegistro + 1
10    Separar línea en campos (id, nombre, edad, carrera)
11    Si nombre = nombreBuscado Entonces
12      Escribir "Registro encontrado:"
13      Escribir "ID: ", id, " | Nombre: ", nombre,
14              " | Edad: ", edad, " | Carrera: ", carrera
15      encontrado ← VERDADERO
16    FinSi
17  FinMientras
18  Si NO encontrado Entonces
19    Escribir "No se encontró el registro."
20  FinSi
21  Cerrar archivo
22 Fin
```

### ¿QUÉ HACE ESTE PSEUDOCÓDIGO?



- 1 Abre el archivo para lectura.
- 2 Pide el nombre a buscar.
- 3 Lee cada registro uno por uno.
- 4 Compara el nombre de cada registro.
- 5 Si lo encuentra, muestra la información.
- 6 Si llega al final sin encontrarlo, informa.

### VARIABLES UTILIZADAS



- **archivo**: archivo de texto abierto para lectura
- **nombreBuscado**: nombre ingresado por el usuario
- **encontrado**: bandera lógica (FALSO / VERDADERO)
- **numeroRegistro**: contador de registros leídos
- **línea**: línea completa leída del archivo
- **id, nombre, edad, carrera**: campos del registro



### VENTAJAS

- Simple de entender
- No requiere estructuras adicionales
- Funciona en cualquier archivo de texto



### DESVENTAJAS

- Lento en archivos grandes
- Debe leer muchos registros hasta encontrar el buscado
- No es eficiente para búsquedas frecuentes



### OBJETIVO

Comprender cómo se implementa el acceso secuencial leyendo los registros uno por uno, desde el inicio del archivo hasta encontrar el buscado o llegar al final.



## 3

## EJEMPLO EN C#

## ACCESO SECUENCIAL EN ARCHIVO DE TEXTO

El siguiente programa busca un alumno por nombre en el archivo "alumnos.txt" leyendo los registros uno por uno.

## ARCHIVO: alumnos.txt



(CSV)

```
1,Ana Pérez,18,Sistemas
2,Carlos López,19,Industrial
3,Luis Martínez,20,Mecatrónica
4,María Gómez,21,Sistemas
5,Pedro Soto,22,Electrónica
6,Julia Ramos,23,Industrial
...
```

## BUSCAMOS

"María Gómez"

**ENCONTRADO**  
en el registro 4

## FORMATO DEL ARCHIVO

ID,Nombre,Edad,Carrera

## CÓDIGO EN C#

```
1 using System;
2 using System.IO;
3
4 class Program
5 {
6     static void Main()
7     {
8         string ruta = "alumnos.txt";
9         string nombreBuscado = "María Gómez";
10        bool encontrado = false;
11        int numeroRegistro = 0;
12
13        using (StreamReader sr = new StreamReader(ruta, System.Text.Encoding.UTF8))
14        {
15            string? linea;
16            while ((linea = sr.ReadLine()) != null)
17            {
18                numeroRegistro++; // contamos registros leídos
19
20                string[] campos = linea.Split(','); // id,nombre,edad,carrera
21                if (campos.Length >= 4)
22                {
23                    string nombre = campos[1].Trim();
24
25                    if (nombre.Equals(nombreBuscado, StringComparison.OrdinalIgnoreCase))
26                    {
27                        Console.WriteLine($"Encontrado en el registro {numeroRegistro}:");
28                        Console.WriteLine($"ID: {campos[0]} | Nombre: {campos[1]} | " +
29                            $"Edad: {campos[2]} | Carrera: {campos[3]}");
30                        encontrado = true;
31                        break; // ya lo encontramos, salimos del bucle
32                    }
33                }
34            }
35
36            if (!encontrado)
37            {
38                Console.WriteLine("No se encontró el registro.");
39            }
40        }
41    }
42 }
```

## 1) Inicialización

Ruta del archivo, nombre a buscar, banderas y contador de registros.

## 2) Lectura secuencial

Leer línea por línea hasta llegar al final del archivo.

## 3) Procesamiento

Separa los campos y compara el nombre.

## 4) Resultado

Si coincide, muestra la información y termina la búsqueda.

## 5) Si no se encontró

Muestra mensaje de no encontrado.

## ¿QUÉ HACE ESTE CÓDIGO?



Abre el archivo para lectura secuencial.



Lee cada línea del archivo una por una.



Separa los campos del registro (ID, Nombre, Edad, Carrera).



Compara el nombre con el buscado (sin distinguir mayúsculas/minúsculas).



Si lo encuentra, muestra el registro y termina.



Si llega al final sin encontrarlo, lo informa.

## EJEMPLO DE SALIDA EN CONSOLA

```
>>> Buscando: María Gómez
Encontrado en el registro 4:
ID: 4 | Nombre: María Gómez |
Edad: 21 | Carrera: Sistemas
```



## NOTA

El acceso secuencial es simple, pero puede ser lento en archivos muy grandes porque revisa registro por registro.



## OBJETIVO

Comprender cómo implementar el acceso secuencial en C# para buscar información en archivos de texto leyendo los registros uno por uno, desde el inicio hasta encontrar el buscado o llegar al final.





# EJEMPLO VISUAL: ARRAY



Archivo secuencial:  
array



Sitio web para interactuar:  
<https://visualgo.net/en/array>

The screenshot shows the Visualgo website interface for the 'array' topic. The browser address bar displays <https://visualgo.net/en/array>. The page header includes the Visualgo logo and navigation tabs: 'en', '/array', 'ARRAY', 'SORTED ARRAY', and 'RQ'. The main content area features a horizontal array of 16 cells. The first seven cells contain the values 0, 0, 5, 6, 44, 78, and 86, while the remaining nine cells are empty. Below each cell is its corresponding index, ranging from 0 to 15, displayed in red text. On the left side, a blue sidebar menu lists various operations: 'Create(M,N)', 'Insert(v)', 'Remove(i)', 'Select', 'Update', and 'Special'. At the bottom of the interface, there is a video player control bar with a progress slider set to 1x, playback buttons (play, pause, stop, next, previous), and links for 'About' and 'Team'.

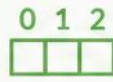
Sitio web para interactuar:  
[Array - VisuAlgo](https://visualgo.net/en/array)

## 1 ¿QUÉ ES?



Un array almacena elementos del mismo tipo en posiciones contiguas.

## 2 ÍNDICES



Cada posición tiene un índice que inicia en 0.

## 3 OPERACIONES



- Insert(v): insertar valor
- Remove(i): eliminar en índice i
- Select: ver valor
- Update: actualizar valor
- Create(M,N): crear array

## 4 USA EL ENLACE



Explora, prueba y visualiza cómo funcionan los arrays en tiempo real.



# ORGANIZACIÓN DE ARCHIVOS

## ACCESO DIRECTO



El **acceso directo** permite ir directamente a la **posición del registro** usando su clave o índice, sin recorrer los registros anteriores. Ideal para búsquedas rápidas en archivos con registros de tamaño fijo.





## 1

# EJEMPLO REAL

## ORGANIZACIÓN DE ARCHIVOS: ACCESO DIRECTO



En el acceso directo, cada registro tiene una posición fija.  
Podemos ir directamente al registro usando su índice (número de posición).



### Ejemplo real:

Archivo de empleados  
(acceso directo por ID)

#### Suposiciones:

- Cada empleado tiene un ID único.
- El archivo tiene espacios fijos para cada ID.
- Para buscar el empleado con ID = 105, vamos directamente a la posición 105.

Buscamos  
ID = 105

| Archivo: empleados.dat (acceso directo por ID) |             |               |               |
|------------------------------------------------|-------------|---------------|---------------|
| Índice (posición)                              | ID Empleado | Nombre        | Puesto        |
| 100                                            | 100         | Ana Pérez     | Analista      |
| 101                                            | 101         | Luis Gómez    | Programador   |
| 102                                            | 102         | María López   | Diseñadora    |
| 103                                            | 103         | Carlos Ruiz   | Soporte       |
| 104                                            | 104         | Sofía Díaz    | Analista      |
| 105                                            | 105         | Pedro Ramírez | Administrador |
| 106                                            | 106         | Laura Torres  | Programador   |
| 107                                            | 107         | Juan Flores   | Soporte       |
| ...                                            | ...         | ...           | ...           |

Acceso  
directo



### ¿CÓMO FUNCIONA?

1 Conocemos  
el ID buscado

105

2 Calculamos la posición  
en el archivo

Posición = ID  
Posición = 105

3 Vamos directamente  
a la posición 105



4 Leemos el registro  
encontrado



### VENTAJAS

- Búsqueda muy rápida.
- Acceso directo sin recorrer registros anteriores.
- Ideal para archivos grandes con registros de tamaño fijo.



**Nota:** El acceso directo requiere que los registros tengan tamaño fijo y un campo clave (ID) único.

## 2

# PSEUDOCÓDIGO

## ORGANIZACIÓN DE ARCHIVOS: ACCESO DIRECTO



El **acceso directo** permite ir directamente a la **posición del registro** usando su clave o índice, sin leer los registros anteriores.

### PSEUDOCÓDIGO

```
1 Inicio
2 Abrir archivo de acceso directo
3 Definir tamañoRegistro
4 Definir claveBuscada
5 Leer claveBuscada
6 Calcular posición = (claveBuscada - claveInicial) * tamañoRegistro
7 Ir a la posición calculada en el archivo
8 Leer registro en esa posición
9 Si registro.clave = claveBuscada Entonces
10     Mostrar "Registro encontrado"
11 SiNo
12     Mostrar "Registro no encontrado"
13 FinSi
14 Cerrar archivo
15 Fin
```

### ¿QUÉ HACE ESTE PSEUDOCÓDIGO?

- 1 Abre el archivo de acceso directo.
- 2 Obtiene la clave (ID) del registro a buscar.
- 3 Calcula la posición exacta usando la clave.
- 4 Se mueve directamente a esa posición.
- 5 Lee el registro y verifica si la clave coincide.
- 6 Muestra el resultado y finaliza.



### VARIABLES UTILIZADAS



- **archivo**: archivo de acceso directo
- **claveBuscada**: clave o ID del registro a buscar
- **claveInicial**: clave del primer registro del archivo
- **tamañoRegistro**: tamaño (en bytes) de cada registro
- **posición**: posición calculada para el acceso
- **registro**: datos leídos en la posición calculada



### NOTA

El acceso directo es muy rápido porque no recorre el archivo; solo calcula la posición y accede al registro deseado.





## 3

# EJEMPLO EN C#

## ORGANIZACIÓN DE ARCHIVOS: ACCESO DIRECTO



En el **acceso directo**, vamos directamente a la **posición del registro** usando su clave (ID), sin leer los registros anteriores.

### CÓDIGO C#

```
1 using System;
2 using System.IO;
3
4 class Empleado
5 {
6     public int Id;
7     public string Nombre;
8     public string Puesto;
9 }
10
11 class Program
12 {
13     static void Main(string[] args)
14     {
15         string ruta = "empleados.dat";
16         int idBuscado = 105;           // ID del empleado a buscar
17         int tamañoRegistro = 100;     // Tamaño fijo de cada registro en bytes
18         long posición = (idBuscado - 1) * tamañoRegistro; // Posición calculada
19
20         using (FileStream fs = new FileStream(ruta, FileMode.Open, FileAccess.Read))
21         {
22             fs.Seek(posicion, SeekOrigin.Begin); // Ir directamente a la posición
23             byte[] buffer = new byte[tamañoRegistro];
24             int bytesLeídos = fs.Read(buffer, 0, tamañoRegistro);
25
26             if (bytesLeídos == tamañoRegistro)
27             {
28                 // Convertir los bytes leídos a objeto Empleado (ejemplo simple)
29                 using (MemoryStream ms = new MemoryStream(buffer))
30                 using (BinaryReader br = new BinaryReader(ms))
31                 {
32                     Empleado emp = new Empleado();
33                     emp.Id = br.ReadInt32();
34                     emp.Nombre = br.ReadString(50).TrimEnd('\0');
35                     emp.Puesto = br.ReadString(30).TrimEnd('\0');
36
37                     Console.WriteLine($"Empleado encontrado: \nID: {emp.Id} \nNombre: {emp.Nombre} \nPuesto: {emp.Puesto}");
38                 }
39             }
40             else
41             {
42                 Console.WriteLine("No se pudo leer el registro en la posición calculada.");
43             }
44         }
45     }
46 }
```



### ¿QUÉ HACE ESTE PROGRAMA?

Busca un empleado por su ID en un archivo de acceso directo (.dat) y se mueve directamente a la posición del registro para leerlo, sin recorrer los anteriores.

### EXPLICACIÓN DEL CÓDIGO

-  Abrimos el archivo de acceso directo para lectura.
-  Obtenemos el ID del registro a buscar.
-  Calculamos la posición usando:  $(ID - 1) * \text{tamañoRegistro}$
-  Nos movemos directamente a esa posición con `Seek()`.
-  Leemos el registro y lo convertimos a objeto `Empleado`.
-  Mostramos el resultado.

### ESTRUCTURA DEL REGISTRO (TAMAÑO FIJO)



| Campo  | Tipo       | Tamaño (bytes) | Descripción         |
|--------|------------|----------------|---------------------|
| Id     | int        | 4              | ID del empleado     |
| Nombre | string(50) | 50             | Nombre del empleado |
| Puesto | string(30) | 30             | Puesto del empleado |
| TOTAL  |            | 100 bytes      |                     |



### NOTAS IMPORTANTES

- Todos los registros deben tener el mismo tamaño.
- La clave (ID) debe ser única.
- Ideal para archivos grandes con consultas rápidas por clave.



**ACCESO  
MUY RÁPIDO**



## 4

# EJEMPLO VISUAL: ACCESO DIRECTO

## Organización de archivos



En el acceso directo, vamos directamente a la **posición** del registro usando su **índice (ID)**, sin recorrer los anteriores.



### IDEA CLAVE

La posición del registro en el array corresponde a su índice (ID).

ID → POSICIÓN

← → ↻ <https://visualgo.net/en/array> ☆ ⋮

**VISUALGO.NET** / en /array ARRAY **SORTED ARRAY** RQ Exploration Mode ▾

ARRAY (índices)

| 0   | 1  | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
|-----|----|---|----|----|----|----|----|----|---|----|----|----|----|----|----|
| -58 | -4 | 8 | 23 | 36 | 81 | 82 | 91 | 95 |   |    |    |    |    |    |    |

**EJEMPLO DE ACCESO DIRECTO**

- 1 Queremos acceder al elemento con índice (ID) = 5  
**ID = 5**
- 2 Vamos directamente a la posición 5  
**POSICIÓN = 5**
- 3 Leemos el valor en esa posición  
**VALOR = 81**

No se recorren los índices 0, 1, 2, 3, 4. Se accede directamente a la posición 5.

**¿CÓMO FUNCIONA?**

- Fórmula para acceso directo:  
**posición = ID**
- En arrays (índices desde 0), el índice (ID) indica exactamente la posición.



### VENTAJAS

- Acceso muy rápido:  $O(1)$
- Ideal para archivos con registros de tamaño fijo y clave única (ID).

### EJEMPLOS DE ÍNDICES Y VALORES

| Índice | 0   | 1  | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9       | ... |
|--------|-----|----|---|----|----|----|----|----|----|---------|-----|
| Valor  | -58 | -4 | 8 | 23 | 36 | 81 | 82 | 91 | 95 | (vacío) | ... |



### RECUERDA

ID único → Posición única  
Acceso directo → Búsqueda inmediata



El acceso directo es muy eficiente para consultas y actualizaciones, ya que evita recorrer todo el archivo.



# ORGANIZACIÓN DE ARCHIVOS EN LA EMPRESA

## ¿CÓMO SE APLICA EN CASOS EMPRESARIALES?



Elegir el método adecuado de acceso a los datos mejora el rendimiento y la toma de decisiones.

### 1. ACCESO SECUENCIAL

Leer registro por registro



#### ¿CÓMO SE APLICA?

Se revisa cada registro uno tras otro, desde el inicio hasta encontrar el dato buscado o llegar al final del archivo.

#### EJEMPLO EMPRESARIAL



##### Inventario de una tienda

Generar reporte de todos los productos para conocer el stock total.

Archivo: inventario.dat

| Pos. | Código | Producto | Stock |
|------|--------|----------|-------|
| 0    | P001   | Teclado  | 25    |
| 1    | P002   | Mouse    | 40    |
| 2    | P003   | Monitor  | 15    |
| ...  | ...    | ...      | ...   |

#### ✓ VENTAJAS

- Sencillo de implementar
- Útil para procesar todos los datos

#### ✗ DESVENTAJAS

- Lento en archivos grandes
- No eficiente para búsquedas específicas

### 2. ACCESO DIRECTO

Ir directamente a la posición



#### ¿CÓMO SE APLICA?

Usa la clave (ID) para calcular la posición del registro y acceder directamente a él, sin recorrer los anteriores.

#### EJEMPLO EMPRESARIAL



##### Consulta de empleado

Obtener rápidamente los datos del empleado con ID = 105.

Archivo: empleados.dat

| Pos. | ID Empleado | Nombre      | Puesto   |
|------|-------------|-------------|----------|
| 0    | 101         | Ana Pérez   | Contador |
| 1    | 102         | Luis Gómez  | Analista |
| 2    | 103         | María López | RRHH     |
| 3    | 104         | Juan Ruiz   | TI       |
| 4    | 105         | Pedro Díaz  | Gerente  |

#### ✓ VENTAJAS

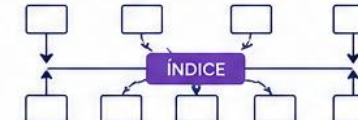
- Muy rápido
- Excelente para consultas frecuentes

#### ✗ DESVENTAJAS

- Requiere clave única
- Puede dejar espacios vacíos

### 3. ACCESO POR ÍNDICE

Usa un índice para localizar



#### ¿CÓMO SE APLICA?

Se consulta primero un índice que indica la posición del registro, y luego se accede directamente al dato.

#### EJEMPLO EMPRESARIAL



##### Sistema bancario

Buscar rápidamente la cuenta de un cliente por su número de cuenta.

Índice (número de cuenta)

| Cuenta | Posición |
|--------|----------|
| 1001   | 0        |
| 1005   | 2        |
| 1010   | 5        |
| 1020   | 8        |
| ...    | ...      |

Archivo: cuentas.dat

| Pos. | Cuenta | Saldo  |
|------|--------|--------|
| 0    | 1001   | Q1,250 |
| 1    | 1003   | Q2,300 |
| 2    | 1005   | Q3,100 |
| 3    | 1007   | Q5,000 |
| 4    | 1009   | Q1,800 |
| ...  | ...    | ...    |

#### ✓ VENTAJAS

- Rápido para búsquedas
- Escalable para grandes volúmenes

#### ✗ DESVENTAJAS

- Requiere espacio para el índice
- El índice debe mantenerse actualizado



#### RECOMENDACIÓN

Analiza el volumen de datos, el tipo de consultas y la frecuencia de acceso para seleccionar el método que mejor se adapte a tu empresa.



#### OBJETIVO

Acceso eficiente a la información = Mejores decisiones + Mayor productividad